Notes — Competitive programming

Adam Martinez[*]
University of Life

1 Bits equalizer

The problem may be solved by performing first a linear scan of the input sequence, keeping track of indices, in two separate lists, for both the `1` bytes in the input sequence that are `0` bytes in the target sequence (the *available* list,) and of the number of `0` or `?` bytes in the input sequence that are `1` bytes in the target sequence (the *required* list.) We make use of the term *list* but any container with decent random access, index-based lookup operations will serve our purposes.

Upon completion of this initial pass, sort the *required* list by indices denoting `0` bytes, and then by indices denoting `?` bytes. Then perform a second pass over the *required* list, and for each index, attempt to swap its corresponding byte in the input sequence for one of the bytes denoted by an element of the *available* list, if any, and remove such index from both the *available* and *required* lists. For each one of these operations, increment the moves counter by `1`. If iteration has not yet finished by the time the *available* list is empty, resolve all `?` and `0` bytes to `1`. For each resolution operation, increment the moves counter by 1.

Compare, in one last linear pass, the (current) input sequence (after all swapping and toggling/setting operations) with the target sequence. Halt iteration as soon as one byte mismatches, and output `-1`, for there is no possible solution. If iteration finishes without any mismatching bytes, then the sequences match, and the number of moves registered thus far should be output.

The total cost of the algorithm should be $O(n)$ for the initial pass, then $O(n \log n)$ to sort the *required* list, then $O(n)$ for the second linear pass comparing the *required* list with the *available* list, and finally one $O(n)$ linear pass to compare the input sequence with the target sequence. For inputs where n ranges between 1 and 100, the asymptotic approximation seems feasible, as it goes for $O(n + n \log n + n) = O(n \log n)$. Considering there are only 200 sample cases per timed program run, ignoring constant factors also seems feasible.

The algorithm falls apart in some test case. Time to figure out what is going on exactly.

Assume the input to be `100?01`, and the target sequence to be `101000`. First, let us establish the lower bound on the number of moves operations for this specific test case.

1. Going from `100?01` to `100001`, we resolve the only `?` byte, which is a mandatory move for any input sequence to even attempt resembling the target sequence.
2. Going from `100001` to `101000`, we swap the two `1` characters and thus compute the final solution; Namely, `2` for a number of moves equivalent to the length of this enumerated list.

The current algorithm attemps to perform these steps in reverse, as it assumes the `?` bytes require resolution anyway, and can be made into any one of `1` or `0` bytes. Thus, it acts first on the constrained parts of the input byte sequence: The `1` and `0` bytes. It prioritizes finding swapping operations instead of toggling operations, such that if some byte index θ denotes a `1` byte in the target sequence where a `0` or `?` byte is found at the same index in the input sequence, and some byte index ω denotes a `0` byte in the target sequence where a `1` byte is found at the same index in the input sequence, it will prioritize swapping these byte indices at the input sequence to avoid performing unnecessary bit toggling operations on existing `0` bytes in the input sequence, as those could very well ruin the input sequence with a larger number of `1` bytes than those present in the target sequence.

[*] staying@never.land

By this heuristic, the algorithm would gather first the two byte indices matching possible swaps, namely byte indices 2 and 5, corresponding with the only "required" `1` and the only "available" `1`. A swap would follow and thus the moves counter would increase by 1. Without any other "required" `1` bytes left, the input byte sequence would perform a linear scan, comparing each of its (current) bytes with the target sequence, and incrementing the moves counter by 1 when encountering a `?` byte in the input sequence.

The missing piece was getting the final linear pass to not only account for the number of `?` bytes, but for the number of `0` bytes in the input sequence that had to be made `1` bytes. This necessity arises in cases where the number of `1` bytes in the target sequence is larger than the combined sum of the number of `?` and `1` bytes in the input sequence, thus requiring toggling operations on some of the latter's `0` bytes.

2 Battleship

The problem may be solved by storing, for each player, only the locations where they keep thei ships. This should take up at most $2(w \times h)$ bytes, for a 2-tuple consisting of the coordinates for the ship. This could potentially overflow system memory if the w were large enough, but the initial iteration should do. Then, assumming player 1 is always the one to start the game, simply emulate each of the, at most, 2000 shot order queries.

Emulation of each of the queries will require an efficient $O(1)$ lookup container to store each player's ship coordinates. A hashset should do just fine. Then, for each of the queries, starting with player 1, check if player 2's collection of ship coordinates contains the shot order in the query, and remove the coordinate from player 2's container if so. Otherwise, switch to assumming the next shot corresponds to player 2, and thus perform the reverse container operations. Whenever a shot order has been determined to be a hit, check the length of the container from which removal has taken place, and halt the game if the container is empty. Check then the length of the attacker's container, and if non-empty, determine the player to have won. Otherwise, it's a draw.

If the winner hasn't been determined before all queries have been processed, then it's a draw.

One consideration the algorithm isn't accounting for is the possibility for a draw after one player has had their navy completely sunk. The initial implementation is not going to put any thought into this, but secrete sample cases likely exploit the fact that the last number of turns that a player took must be repeated by the other player prior to determining a winner, irrespective of whether the last shot completely sank all of the other player's ships.

The current implementation considers the above case, but is lacking in some respect. The problem statement seems ambiguous in its reach; It is said that each player ought have the same number of turns, but on special consideration is put into whether a game may end abruptly with one player having fewer turns than another player.

This is made clear by some example test case whereby the sequence of shots is assumed to follow this correspondence:

1. Player 1 hits player 2
2. Player 1 hits player 2

- Player 2 is left without any other vessels, so following the first point on the number of consecutive turns that a player may take, player one is owed another turn, even if no ships remain on its navy.

1. Player 1 falters.
2. Player 1 hits player 2
3. Player 1 hits player 2
4. Player 1 hits player 2

- Player 2 follows up with another move on player 1, even if neither of them have any ships left.

1. Player 2 falters.

At this point, if we assume that player 2's ships have all been sinked, and thus control should be handed back to it, game rules dictate that its turn should be made up of at least three more turns. This should only really affect the time complexity of the implemented algorithm and not the final solution, for the problem is unaffected by further turns as no more ships remain on any one side (even though the time complexity here would be completely ruined.)

If we assume that game rules implicitly dictate that the only possibility for turn-taking extensions is that of satisfying both initial requirements, then surely there's a point where one player ought

abandon the game without having taken the same set of turns, for otherwise termination would never be reached.

Resolution of such a situation would be non-trivial, especially considering the fact that if by the end, player 2 is bound to same number of turns as player 1, then surely player 1 is bound to the same number of turns as player 2. But that inherently goes against the rules for turn swapping between players, which dictate that no player ought hold the turn if (1) they haven't hit the opposing player, **and**, (2) the other player has some ship left unsinked in its navy.

If these rules held, then termination would be possible, for indeed only a single turn would be awarded to a player, a "mercy turn" of sorts; The rest would have to follow from that player having satisfied both requirements and thus have "won" the right to have another shot.

This certainly implies that each player ought have at most one turn, and quite possibly that turns are not incremented by the number of shots that player may take at a time, but rather by the act of swapping from one player to another.

Turn-taking logic turned out to be unrequited, and indeed, all secret sample cases run without issues except the same failing subset as presented before. The issue must lie somewhere else, for otherwise one of the implemented strategies would've worked differently, but they all seem to be aligned in experimental behavior.

The issue may be in a small detail from the second paragraph of the problem statement. It states that the second player may get another turn even if their entire navy is sunk, but my program assumed that to mean that any player is getting another turn if they sink the entire navy of the opposing player.

But it seems like the only player that gets another turn even if its navy is fully sunk is player two, and not player one, were the latter to have all its ships sunk.

The final solution should thus be to only ever allow switching players without trigerring the `fail` flag for input-only processing, upon reaching one of the existing states, but now only for player 1. That should allow reusing the same algorithm, but adding to it a slight change related to functionality once one of the current states where the `fail` flag is modified actually hits.

This solved the problem.

3 Tic-tac-toe

The problem seems to be akin to a simulation problem, except the simulation steps are not given. Instead, one is expected to either precompute all possible scenarios and evalute whether any one of them matches the end result, or otherwise perform an in-place simulation as data is read in.

Clearly, there are no limits on the memory that may be used at once for the purposes of reading in some input data, as sample test cases per run have an upper bound of 150. For each of the sample cases, a single 3×3 grid is layed out, which shouldn't put a constraint on reading in all of `stdin` at once.

The issue then lies in determining whether the final scenario can be reached in a game of tic-tac-toe. The first thing that comes to mind is the possibility of using dynamic programming, and more specifically using a bottom-up approach where not all states are pre-computed. This should allow considering the first of the pieces in the board, and compute the next set of possible states at that point. If the next movement that we read in from the input data set turns out to be one of the states we just computed, then the algorithm may **not** halt, and we can repeat the same operation. If the algorithm cannot determine which of the next-computed states is the one being considered at present, then it should terminate.

If the algorithm terminates prior to having processed the complete input for some sample test case, then it hasn't been capable of determining a possible scenario in tic-tac-toe that could be reached.

Thinking it some more, maybe the solution is more trivial than it may seem at first glance. If we consider instead that any one given situation is possible, so long as the number of `X`-marked cells is only one unit larger than the number of `0`-marked cells, then we can more simply determine whether the state can be reached or not by checking for the number of `X`- and `0`-marked cells in the grid under consideration.

There are multiple scenarios to take into consideration, though, as situations like start game have a different layout than throughout the rest of the play.

Let us thus consider the different set of possible states.

- Upon starting the game, the grid is empty and thus corresponds with a valid state.
- Upon the first player, namely the player marking cells with `X`, taking their turn, the grid is left with a single, `X`-marked cell.
- Beyond this, the game can only be in one of two possible states; One where the total number of `0`-marked cells is equal to the number of `X`-marked cells, or one where it is strictly one unit smaller than the number of `X`-marked cells.

  The first of these situations would correspond with having finished player `0`'s turn, and having player `X`'s turn come up next (i.e. the sample test case corresponds with that of a snapshot right before player `X` is about to play.)

  The latter would take place if player `X` were to have just finished its turn, and thus the snapshot showcased by the test sample would correspond with the state of the grid right before player `0` were to take its turn.

Upon further inspection, there's a possibility we haven't considered. Suppose an end game state is reached, where the number of `X`-marked cells is larger than the number of `0`-marked cells. At this point, no more states are valid, but the grid may or may have not been marked in its entirety. Assume the latter scenario. A possible but invalid state would be reached by adding another `0`-marked cell, which would correspond with one of the valid states, namely the one where player `0` has just performed its move. If we assume the current state to be the one showcased in the test sample, then by the above-enumerated rules, the grid would be deemed to be in an valid state, even when clearly it has gone past end game.

Solving this should be fairly simple; Keeping track of the marked locations for each player, and adding another check while processing the input data set for one of eight possible end game arrangements should do just fine. Note we speak of eight possible end-game arrangements because a 3×3 grid only has 8 different straight line sequences that could be filled by one of the two players.

An ideal data structure to peform those lookups would be a hashset where a set of locations can be checked in $O(1)$. Because the locations to check are comprised of only $8 \times 3 = 24$ elements, and it is to be made for upwards of 150 games, there should be no issues with performing $150 \times 24 \times 2 = 7,200$ lookups at worst.

The solution seems to be missing something. Either the algorithm is being too stringent in the allowed states, or it's missing some erroneous case. The problem is that the problem seems too simple; the only allowed states are those in which player `X` has the same number of occupied grid cells as player `0`, or otherwise has one more marked position. This is then stacked with the fact that if any one of `X` or `0` contain winning sets of positions larger than 1, the game has reached end game state.

This latter condition is checked by first computing the cardinality of such sets, and then ensuring that they're either both 0, or that only *one* of them is equal to one. This would align with the states corresponding with having no winner in the input grid, or with having a single winner, which would represent an end game state.

It's quite possible that the problem is actually expecting the algorithm to perform extensive input validation. This is likely because the initial problem statement mentions the set of conditions that make a game valid, among which we find the grid size. This is not further remarked in the input description, which may mean that the one case that is not being handled is the possibility for the grid to be smaller than the expected 3×3 grid. This is the only real case that the current solution is not taking into consideration.

But it turns out that is not the right approach either. The current implementation performs complete input validation, except for row number validation because this is actually always accurate based on the input description. This issue must lie somewhere else.

The problem has finally been solved and it was not an *Eureka* nor a *Hmm, that's funny* moment (meaning it did not come from a one-off idea nor a long session of work, but rather of a (possibly idiotic) refactoring.) The solution is odd, as it surged merely from an attempt to further "functionalize" my code (make it beautifully functional and code-golfy.) The solution seemed to be in performing further input validation, such that by the time the program was performing parsing of possible winning positions, the algorithm should've exitted early as soon as more than a single winning position was reached (regardless of the player that had reached such a position.)

This aligns with the initial algorithm design that performed a similar input validation at a later stage to check that a state beyond end game was not being considered possible. The reason why (I, as of writing this, believe) this refactoring shouldn't have yield the correct solution is that such a check was already being performed right after parsing all winning positions (ensuring only that both players couldn't have gotten winning positions.) This should've been caught at the stage where pattern matching makes sure the ratio of winning positions for player `X` is 1 against 0 from player `0`, or the other way around. But it didn't seem to catch on, and as it turns out, the solution was in performing a full lazy evaluation of the input sample, by breaking early out of the winning position parsing stage as soon as a win event was determined to have taken place (irrespective of player) after another win was determined to have already taken place in the pattern-matching state machine.

4 Divide by 100

The problem is fairly simple, but making it time-efficient is a bit tricky. Initially, the algorithm attempted to perform a linear scan over both inputs, namely N and M. This makes sense for the former, as we always require knowing which number we're working with, and indeed, there is no non-trivial way of determining the number of trailing zeroes in N without performing a linear scan over its digits. However, the same cannot be said for M. In this instance, we can perform a parallel read between the input bytes of this number and the bytes of N such that string processing happens as input is read. This should also allow, in some worst-case scenarios, to avoid a buffer overflow due to the extreme upper bound set in the problem statement for M.

Currently, implementation efforts are focused on having the internal buffer for N be a double-ended queue, such that we can perform efficient push and pop operations on both ends of the buffer, as we may need to both add leading zeroes and add a leading dot to the number, depending on the number of zeroes in M. Combining the mandatory linear scan over M with parllel insertions at the front and back of the buffer for N should allow us to perform the necessary string processing efficiently, and thus solve the problem within the time limits that the initial submittion failed to meet.